

Concurrencia (hilos), manejo de archivos NIO y serialización

Qué es y qué problema resuelve

Son tres piezas del JDK que casi siempre aparecen juntas en un mismo tipo de programa: uno que tiene varias tareas independientes que procesar (por ejemplo, veinte pedidos), quiere aprovechar más de un núcleo para hacerlo, y al terminar necesita guardar el resultado en disco de forma que sobreviva a que el programa se cierre.

- **Hilos (ExecutorService):** Los hilos resuelven un problema que tiene el paradigma al pasar o correr varias veces el servicio a diferentes usuarios, esto reparte el trabajo entre cada hilo. Un ExecutorService mantiene un pool fijo de hilos reutilizables y reparte las tareas entre ellos.
- **El problema de la concurrencia:** Es cuando dos hilos modifican el mismo archivo, esto se soluciona al tener un orden de jerarquía o que cuando un hilo modifique algo, este se bloquee para los demás.
- **Archivos (java.nio.file.Files):** Resuelven la persistencia de cada resultado individual como texto plano, de forma que un handle de archivo nunca se quede abierto por accidente aunque algo falle a medio camino.
- **Serialización Java:** Es convertir un objeto completo a una secuencia de bytes que se puede guardar en disco y, después, reconstruirlo.

Dónde se ve en el código

Todo vive en procesador-pedidos/src/main/java/com/academymty/concurrencia/:

- **Pedido.java:** El dato de entrada (un record simple, no serializable a propósito: no es lo que se persiste).
- **GeneradorPedidos.java:** Fabrica 20 pedidos de prueba con datos reproducibles.
- **DemoCondicionCarrera.java:** La demostración explícita del problema de concurrencia: la clase corre el mismo incremento con un int normal (inseguro) y con AtomicInteger (seguro), y compara los dos resultados.
- **ProcesadorPedidos.java:** El orquestador: crea un ExecutorService con Executors.newFixedThreadPool, manda cada pedido a procesar en paralelo, usa un AtomicInteger para contar éxitos/fallos y un ConcurrentLinkedQueue para llevar la lista de archivos generados sin condición de carrera. Cada tarea escribe su archivo con Files.newBufferedWriter(...) dentro de un try-with-resources.
- **ResumenProcesamiento.java:** La clase que sí se serializa: implementa Serializable, declara serialVersionUID y tiene un campo transient.

- **Main.java:** Corre las tres partes en orden: la demo de la carrera, el procesamiento con hilos y archivos, y por último ObjectOutputStream/ObjectInputStream para guardar y releer el resumen.

Qué pasa si no lo usas

- **Sin ExecutorService (un hilo por tarea, sin pool):** Con pocas tareas no se nota, pero en cuanto la cantidad crece, crear un Thread por cada uno, agota memoria e hilos del sistema operativo; el programa se vuelve más lento que procesar todo secuencialmente.
- **Sin proteger el dato compartido:** DemoCondicionCarrera lo mide directamente 8 hilos incrementando el mismo int 2,000 veces cada uno deberían dar 16,000, pero sin protección el resultado cae a poco más de 2,000. Se pierden más del 80% de los incrementos, y el número cambia en cada corrida porque el entrelazado de los hilos no es determinista. Con AtomicInteger el resultado es 16,000 siempre.
- **Sin try-with-resources al escribir archivos:** Si algo lanza una excepción a media escritura, el BufferedWriter se queda abierto. Con pocos archivos no pasa nada visible; con miles, el proceso se queda sin handles disponibles y empieza a fallar al abrir archivos nuevos que ni siquiera tienen que ver con el error original.
- **Sin Serializable:** ObjectOutputStream.writeObject(resumen) lanza NotSerializableException de inmediato. No hay forma de guardar el objeto sin ese contrato.

Cómo correrlo

```
Bash
cd "01-core avanzado/procesador-pedidos"
mvn compile exec:java # o: ./scripts/correr.sh
```